

Algorithm for file updates in Python

Project description

As a security analyst, controlling access to restricted content is critical for maintaining an organization's network security posture. In this project, I developed an automated Python algorithm to parse and maintain a security log file (`allow_list.txt`) containing IP addresses authorized to access restricted information.

By automating file parsing and list manipulation in Python, the algorithm dynamically checks for revoked access permissions and updates the allow list to reflect current security policies without manual file editing.

Key Objectives & Steps

- **File Import & Parsing:** Created context managers (with statements) to safely read and load raw IP text files into Python environments.
- **Data Transformation:** Used string manipulation methods like `.split()` to convert raw log data into manageable Python lists for element-by-element iteration.
- **Conditional Removal Algorithm:** Implemented nested control flow logic (for loops and if conditional statements) paired with the `.remove()` method to filter out revoked IP addresses stored in a removal list.
- **File Updates:** Re-assembled updated lists into standardized text strings using `".join()"` and safely rewrote the original security log files in write mode ("w").
- **Modular Code Structure:** Encapsulated the entire parsing algorithm into a reusable Python function (`update_file()`) to automate future access-control list updates.

Open the file that contains the allow list

```
# allow_list.txt initiation

ip_addresses = "192.168.218.160 192.168.97.225 192.168.145.158
192.168.108.13 192.168.60.153 192.168.96.200 192.168.247.153 192.168.3.252
192.168.116.187 192.168.15.110 192.168.39.246"

with open("allow_list.txt", "w") as file:
    file.write(ip_addresses)
```

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer
allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40",
"192.168.58.57"]

# First line of `with` statement
# READ WITH STATEMENT

with open(import_file, "r") as file:
    pass
```

Read the file contents

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer
allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40",
"192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
# READ WITH STATEMENT

with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable
    named `ip_addresses`

    ip_addresses = file.read()
```

```
# Display `ip_addresses`
```

```
print(ip_addresses)
```

```
192.168.218.160 192.168.97.225 192.168.145.158 192.168.108.13  
192.168.60.153 192.168.96.200 192.168.247.153 192.168.3.252  
192.168.116.187 192.168.15.110 192.168.39.246
```

Convert the string into a list

```
# Assign `import_file` to the name of the file
```

```
import_file = "allow_list.txt"
```

```
# Assign `remove_list` to a list of IP addresses that are no longer  
allowed to access restricted information.
```

```
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40",  
"192.168.58.57"]
```

```
# Build `with` statement to read in the initial contents of the file  
# READ WITH STATEMENT
```

```
with open(import_file, "r") as file:
```

```
    # Use `.read()` to read the imported file and store it in a variable  
    named `ip_addresses`
```

```
    ip_addresses = file.read()
```

```
# Use `.split()` to convert `ip_addresses` from a string to a list
```

```
ip_addresses = ip_addresses.split()
```

```
# Display `ip_addresses`
```

```
print(ip_addresses)
```

```
['192.168.218.160', '192.168.97.225', '192.168.145.158', '192.168.108.13',  
'192.168.60.153', '192.168.96.200', '192.168.247.153', '192.168.3.252',  
'192.168.116.187', '192.168.15.110', '192.168.39.246']
```

Iterate through the remove list

```
# Assign `import_file` to the name of the file  
  
import_file = "allow_list.txt"  
  
# Assign `remove_list` to a list of IP addresses that are no longer  
# allowed to access restricted information.  
  
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40",  
"192.168.58.57"]  
  
# Build `with` statement to read in the initial contents of the file  
# READ WITH STATEMENT  
  
with open(import_file, "r") as file:  
  
    # Use `.read()` to read the imported file and store it in a variable  
    # named `ip_addresses`  
  
    ip_addresses = file.read()  
  
# Use `.split()` to convert `ip_addresses` from a string to a list  
  
ip_addresses = ip_addresses.split()  
  
# Build iterative statement  
# Name loop variable `element`  
# Loop through `ip_addresses`  
  
for element in ip_addresses:  
  
    # Display `element` in every iteration
```

```
print(element)
```

```
192.168.218.160
192.168.97.225
192.168.145.158
192.168.108.13
192.168.60.153
192.168.96.200
192.168.247.153
192.168.3.252
192.168.116.187
192.168.15.110
192.168.39.246
```

Remove IP addresses that are on the remove list

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer
allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40",
"192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
# READ WITH STATEMENT

with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable
    named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

```

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

        if element in remove_list:

            # then current element should be removed from `ip_addresses`

            ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)

```

```

['192.168.218.160', '192.168.145.158', '192.168.108.13', '192.168.60.153',
'192.168.96.200', '192.168.247.153', '192.168.3.252', '192.168.116.187',
'192.168.15.110', '192.168.39.246']

```

Update the file with the revised list of IP addresses

```

# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer
allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40",
"192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
# READ WITH STATEMENT

```

```
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable
    # named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into
# the text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)

print(ip_addresses)
```

```
192.168.218.160 192.168.145.158 192.168.108.13 192.168.60.153
192.168.96.200 192.168.247.153 192.168.3.252 192.168.116.187
192.168.15.110 192.168.39.246
```

Summary

- Automated Access Control: Developed a Python script to dynamically manage and maintain security access lists (`allow_list.txt`) by removing unauthorized IP addresses.
- Syntax, Functions, & Keywords Explained:
 - `with` Statement & `open()` Function: Used `with open(import_file, "r") as file:` and `with open(import_file, "w") as file:` to open and handle the file. The `with` statement serves as a context manager that automatically closes the file after operations complete, preventing memory leaks and file corruption. The `"r"` mode opens the file for reading, while `"w"` opens it for writing (overwriting existing content).
 - `.read()` & `.write()` Methods: Applied `.read()` inside the read context manager to convert the text file's entire content into a single string variable (`ip_addresses`). Later, I used `.write(ip_addresses)` inside the write context manager to write the updated, stringified IP list back to the physical file.
 - `.split()` Method: Called `.split()` on the raw `ip_addresses` string. By default, `.split()` separates a string by whitespace (spaces, tabs, newlines) and converts it into a list of individual IP address strings, allowing list manipulation.
 - `for` Loop & `if...in` Control Flow: Implemented an iterative `for element in ip_addresses:` loop to pass through each IP in the list. Inside the loop, the `if element in remove_list:` conditional keyword checks whether the current IP exists inside the target removal list.
 - `.remove()` Method: Executed `ip_addresses.remove(element)` inside the conditional block to mutate the `ip_addresses` list in place, deleting any matching IP address that is no longer permitted access.
 - `" ".join()` Method: Re-assembled the modified list into a single space-separated string via `" ".join(ip_addresses)`, making the data compatible with `.write()`.
- **Key Takeaway:** Combined core Python file handling, list operations, and control structures to transform a manual administrative task into a reliable, automated security tool.